Day 3 - Doctor Management Module

Date

2026-08-24

Overview

Day 3 introduced the Doctor Management module, the second core entity of the Clinic Management System. This includes a complete CRUD system for doctors, specialization-based filtering, search functionality, pagination, and full Thymeleaf web interface.

Technology Stack Used

Technology	Version	Purpose
Java	26	Programming Language
Spring Boot	4.1.0	Backend Framework
Spring Data MongoDB	4.1.0	Database Integration
MongoDB	Latest (local)	NoSQL Database
Thymeleaf	4.1.0	Server-side HTML templating
Bootstrap	5.3.2 (CDN)	UI Styling
Spring Validation	4.1.0	Request DTO validation
SpringDoc OpenAPI	2.8.0	API Documentation
Lombok	1.18.46	Boilerplate code reduction
Maven	Bundled with IntelliJ	Build & Dependency Management
Git	Latest	Version Control

What Was Built Today

1. Specialization Enum (enums/)

`Specialization.java`

- `CARDIOLOGY`
- `DERMATOLOGY`
- `NEUROLOGY`
- `PEDIATRICS`
- `GENERAL_MEDICINE`
- `ORTHOPEDICS`

2. Doctor Entity (entity/)

`Doctor.java` - MongoDB document mapped to `doctors` collection:

Field	Type	Description
`doctorId`	`String`	Custom ID (e.g., `DOC-A1B2C3D4`)
`name`	`String`	Doctor full name
`email`	`String`	Contact email
`phone`	`String`	Contact phone
`specialization`	`Specialization`	Enum: CARDIOLOGY, DERMATOLOGY, etc.
`qualification`	`String`	Medical degrees
`experience`	`Integer`	Years of experience
`availability`	`String`	Working hours schedule
`createdAt`	`LocalDateTime`	Auto-populated timestamp
`updatedAt`	`LocalDateTime`	Auto-populated timestamp

Annotations: `@Document`, `@Id`, `@CreatedDate`, `@LastModifiedDate`, `@Data`, `@Builder`, `@NoArgsConstructor`, `@AllArgsConstructor`

3. DTOs (dto/)

`DoctorRequestDto.java` - Incoming API requests:

- Validation: `@NotBlank` on name, email, phone, qualification
- `@Email` on email
- `@Size(min=10, max=15)` on phone
- `@NotNull` on specialization, experience
- `@Positive` on experience
- Lombok: `@Data`, `@Builder`, `@NoArgsConstructor`, `@AllArgsConstructor`

`DoctorResponseDto.java` - Outgoing API responses:

- Mirrors all entity fields including timestamps
- Lombok: `@Data`, `@Builder`, `@NoArgsConstructor`, `@AllArgsConstructor`

4. Custom Exception (exception/)

`DoctorNotFoundException.java`

- Extends `RuntimeException`
- Thrown when doctor ID does not exist

5. Repository Layer (repository/)

`DoctorRepository.java` - Extends `MongoRepository<Doctor, String>`:

- `findByEmail(String email)` - Duplicate checking

- `findBySpecialization(Specialization specialization)` - Filter by specialization
- `findBySpecialization(Specialization, Pageable)` - Paginated specialization filter
- `findByNameContainingIgnoreCase(String name)` - Name search
- `findByNameContainingIgnoreCase(String name, Pageable)` - Paginated name search
- `findAll(Pageable)` - Paginated list

6. Service Layer (service/)

`DoctorService.java` - Interface defining:

- `createDoctor()`, `getAllDoctors()`, `getDoctorById()`, `updateDoctor()`, `deleteDoctor()`
- `getDoctorsBySpecialization()`, `searchDoctorsByName()`
- `getAllDoctorsPaginated()`, `getDoctorsBySpecializationPaginated()`

`DoctorServiceImpl.java` - Implementation:

- `createDoctor()` - Generates `DOC-XXXX` ID, checks duplicate email, sets timestamps
- `getDoctorById()` - Throws `DoctorNotFoundException` if not found
- `updateDoctor()` - Updates all fields, sets new `updatedAt`
- `deleteDoctor()` - Verifies existence before deletion
- Pagination methods using `PageRequest.of(page, size)`
- `mapToResponseDto()` private helper

7. REST API Controller (controller/)

`DoctorRestController.java` - Base path: `/api/doctors`

Method	Endpoint	Description
`POST`	`/api/doctors`	Create doctor
`GET`	`/api/doctors`	List all (optional pagination)
`GET`	`/api/doctors/{id}`	Get by ID
`PUT`	`/api/doctors/{id}`	Update doctor
`DELETE`	`/api/doctors/{id}`	Delete doctor
`GET`	`/api/doctors/specialization?specialization=`	Filter by specialization
`GET`	`/api/doctors/search?name=`	Search by name

8. Thymeleaf Controller (controller/)

`DoctorViewController.java` - Web UI endpoints:

- `GET /doctors` - List all doctors
- `GET /doctors/new` - Show create form
- `POST /doctors/create` - Handle form submission
- `GET /doctors/{id}` - View doctor details

- `GET /doctors/{id}/edit` - Show pre-filled edit form
- `POST /doctors/{id}/update` - Handle update submission
- `GET /doctors/{id}/delete` - Delete with confirmation
- Passes `specializations` array to form templates for dropdown

9. Thymeleaf Templates (templates/doctors/) `list.html` - Doctor

listing page:

- Bootstrap table with dark header
- Badge for specialization display
- Action buttons: View, Edit, Delete
- Empty state message
- Total doctor count badge
- "+ Add Doctor" button `form.html` - Add/Edit form:
- Dynamic title and submit button
- Fields: Name, Email, Phone, Specialization (dropdown from enum), Qualification, Experience, Availability
- Cancel button returns to list
- Pre-populated for edit mode `detail.html` - Doctor detail view:
- Info card with Doctor ID badge
- Table with all fields
- Formatted timestamps
- Back, Edit, Delete buttons
- Delete confirmation dialog

10. Updated Navigation (templates/)

All pages updated with Doctors link in navbar:

- `index.html` - Added Doctors dashboard card, updated buttons
- `patients/list.html` - Added Doctors nav link
- `patients/form.html` - Added Doctors nav link
- `patients/detail.html` - Added Doctors nav link

Testing Performed

REST API Tests

1. **Create Doctor**

```
POST http://localhost:8080/api/doctors
{
    "name": "Dr. Ahmed Khan",
    "email": "ahmed.khan@example.com",
    "phone": "03001234567",
    "specialization": "CARDIOLOGY",
    "qualification": "MBBS, FCPS (Cardiology)",
    "experience": 15,
    "availability": "Mon-Fri 9AM-5PM"
}
```

- Result: `201 Created` with `doctorId: "DOC-XXXX"`

2. **Filter by Specialization**

```
GET http://localhost:8080/api/doctors/specialization?specialization=CARDIOLOGY
```

- Result: List of cardiologists only

3. **Paginated List**

```
GET http://localhost:8080/api/doctors?page=0&size=2
```

- Result: Page object with metadata

Thymeleaf UI Tests

- `/doctors` -> Table renders with all doctors
- `/doctors/new` -> Form with specialization dropdown
- `/doctors/{id}` -> Detail page with all fields
- `/doctors/{id}/edit` -> Pre-filled form
- Delete -> Confirmation dialog -> Removed from list

Challenges Faced & Solutions

Challenge	Solution
`doctors/list.html` template not found (500 error)	Created missing `doctors/` folder under `templates/` manually in IntelliJ, added 3 HTML files
Thymeleaf specialization dropdown empty	Passed `Specialization.values()` from controller to model
Old test data with null IDs	Cleared MongoDB collection and recreated via REST API

Git Activity

- **Repository:** `https://github.com/abdul15x/clinic-management-system`
- **Branch:** `main`
- **Commit:** `feat: add doctor management with specialization search and Thymeleaf forms`
- **Files pushed:** New entity, DTOs, exception, repository, service, controllers, 3 HTML templates, updated navbar on all pages

---

Files Created/Updated Today

New Files

```
src/main/java/com/clinic/enums/Specialization.java
src/main/java/com/clinic/entity/Doctor.java
src/main/java/com/clinic/dto/DoctorRequestDto.java
src/main/java/com/clinic/dto/DoctorResponseDto.java
src/main/java/com/clinic/exception/DoctorNotFoundException.java
src/main/java/com/clinic/repository/DoctorRepository.java
src/main/java/com/clinic/service/DoctorService.java
src/main/java/com/clinic/service/DoctorServiceImpl.java
src/main/java/com/clinic/controller/DoctorRestController.java
src/main/java/com/clinic/controller/DoctorViewController.java
src/main/resources/templates/doctors/list.html
src/main/resources/templates/doctors/form.html
src/main/resources/templates/doctors/detail.html
```

Updated Files

```
src/main/resources/templates/index.html
src/main/resources/templates/patients/list.html
src/main/resources/templates/patients/form.html
src/main/resources/templates/patients/detail.html
```

What's Next (Day 4 Preview)

- User Entity with roles (ADMIN, DOCTOR, RECEPTIONIST)
- Spring Security setup
- Password encryption with BCrypt
- Registration and Login endpoints
- Basic security configuration

How to Run This Version

1. Ensure MongoDB is running on `localhost:27017`
2. Open project in IntelliJ IDEA
3. Set Project SDK to Java 26

4. Enable Annotation Processing
5. Run `ClinicManagementSystemApplication.java`
6. Access:
    - Web UI: `http://localhost:8080/`
    - Patients: `http://localhost:8080/patients`
    - Doctors: `http://localhost:8080/doctors`
    - REST API: `http://localhost:8080/api/doctors`